

Compressed NN for Diabetes detection on FPGA with hls4ml

Vangos Theodoros
03765
tvangos@uth.gr

Oikonomopoulos Vasilis
03406
voikonom@uth.gr

I. ABSTRACT

This project investigates the use of a neural network for the prediction of diabetes using a hardware-oriented machine learning workflow. A dataset with information about diabetes patients was used to train a small feedforward neural network for binary classification. After evaluating the model's performance in software, the trained network was converted into a hardware implementation using the `hls4ml` framework, which generates High-Level Synthesis code for a FPGA board, with the results demonstrating that a compact neural network can indeed achieve competitive classification performance while being efficiently translated into hardware.

II. INTRODUCTION

Machine learning has become an increasingly important tool in the last few years in healthcare due to the assistance that it can offer with the diagnosis and management of diseases. Diabetes is one of the most widespread chronic diseases worldwide, making it an appropriate application for machine learning-based classification techniques. Thus, by analyzing patient health data, neural networks can easily learn complex relationships between clinical features and the likelihood of diabetes.

For this project, we developed a feedforward neural network, also known as a multilayer perceptron, using the Open ML Diabetes dataset. The network was trained to perform binary classification, predicting whether a patient does or doesn't have diabetes based on eight medical attributes. Its architecture consisted of two fully connected hidden layers with ReLU activation functions, batch normalization, dropout layers for regularization, and a sigmoid output layer.

After training and evaluating the model in TensorFlow/Keras, the next step was implementing it in hardware using the `hls4ml` framework. This framework translates trained neural networks into High-Level Synthesis code, allowing our model to run on a FPGA with low latency and efficient resource utilization. The hardware implementation was compared with the original software model to verify that the conversion preserved its predictive performance and did not equal to a coin flip while also enabling hardware acceleration.

So for this project apart from the installation of python and Linux we primarily needed these libraries for the setup:

- TensorFlow / Keras – for the neural network construction and training
- hls4ml – for the conversion of the trained model to HLS for FPGA deployment
- Scikit-learn – for dataset loading, preprocessing, train/test splitting, class weighting, and evaluation metrics
- NumPy – for the numerical operations and reproducibility
- Matplotlib – for plotting the ROC curves and performance comparison

Finally the board we used for the timing and utilization measurements was the Alveo U250 which is one of the few supported FPGAs by the `hls4ml` framework.

III. DATASET AND PREPROCESSING

A. Dataset Overview

For our project we used the OpenML Diabetes version 7 dataset, which contains medical records for 768 patients. Each patient record consists of eight continuous input features:

- Pregnancies
- Glucose
- Blood Pressure
- Skin Thickness
- Insulin
- Body Mass Index
- Diabetes Pedigree Function
- Age

The target variable, `TestedPositiveForDiabetes`, is binary, where a value of 1 indicates that the patient tested positive for diabetes, while a value of 0 indicates a negative diagnosis. Consequently, the neural network was designed with eight input neurons and a single output neuron.

B. Missing Value Handling

Although the dataset does not explicitly contain missing values, several physiological measurements contain zero values that represent missing or unavailable data rather than valid measurements. In particular, the `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, and `BMI` features contained zero values which are biologically impossible for a living patient.

Leaving these values unchanged introduced misleading information into the training process and reduced the predictive performance of our neural network. Therefore, all zero values

in these five attributes were treated as missing values by replacing them with NaN.

Before, the number of missing entries represented by zeros was:

Feature	Missing Values
Insulin	374
Skin Thickness	227
Blood Pressure	35
BMI	11
Glucose	5

TABLE I
NUMBER OF MISSING VALUES REPRESENTED BY ZEROS BEFORE
PREPROCESSING.

The missing values were all replaced with the mean value of each corresponding feature, thus preserving all patient records while also providing reasonable estimates for the missing measurements.

C. Feature Scaling

The input variables span significantly different numerical ranges. For example, the `DiabetesPedigreeFunction` typically ranges between 0 and 2, whereas `Insulin` values may exceed 300. Such differences can negatively affect the convergence of the optimization algorithms we used.

To ensure that all features contributed equally during training, the input data were standardized using the `StandardScaler` from Scikit-learn, resulting in a faster and more stable neural network training process.

D. Train-Test Split and Class Imbalance

The preprocessed dataset was divided into training and testing subsets using an 80:20 split with a fixed random seed (`random_state = 42`) to ensure reproducibility of the results.

Inspection of the training labels revealed an imbalanced class distribution consisting of 404 negative (non-diabetic) cases and 210 positive (diabetic) cases, corresponding to an approximate 2:1 ratio.

Because neural networks can become biased toward the majority class, balanced class weights were computed and applied during training. This weighting strategy we chose assigns greater importance to the minority class when computing the loss function, thus helping our model improve its ability to correctly identify diabetic patients.

IV. THE NEURAL NETWORK

In order to achieve a low latency design on the FPGA, we implemented a multi-layer perceptron (MLP) optimized later via Quantization-Aware Training (QAT). The network is trained using the `Keras` and then the `QKeras` (Quantized `Keras`) library, which allowed us in the optimization phase to create fixed-point arithmetic inputs, outputs, weights, biases etc. This ensured that our model learned the optimal weights with reduced precision but without any major precision loss.

A. Model Structure

The neural network is designed as a sequential feed-forward architecture, for binary classification. It consists of an input layer, two intermediate hidden blocks, and an output block. The specific sequence of operations is defined as follows:

- **A simple Input Layer of size 8x1 (`fc1_input`)**
- **First Hidden Block:**
 - **Dense Layer (`fc1`):** A fully connected layer computing the dot product of the input vector and the learned weight matrix ($W \times X$).
 - **Batch Normalization (`batch_normalization`):** This layer forces the raw numbers coming out of the dense layer to shrink back down into a safe, small range so they fit into our hardware without causing an overflow.
 - **Activation (`relu1`):** A Rectified Linear Unit (ReLU) that applies a non-linear transformation with the function $f(x) = \max(0, x)$.
 - **Dropout:** a rate of 0.2 (randomly dropping 20% of the neurons) was applied. Because Dropout is strictly a training-phase feature, it consumes zero FPGA resources during synthesis.
- **Second Hidden Block:**
 - **Dense Layer (`fc2`):** A subsequent fully connected layer
 - **Batch Normalization**
 - **Activation (`relu2`):** A second ReLU activation function.
 - **Dropout**
- **Output Block:**
 - **Dense Layer (`output`):** A final fully connected layer that reduces the output to a single output neuron.
 - **Activation (`sigmoid`):** A sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$, producing a final probability score $p \in [0, 1]$ for binary classification.

B. Design

The design and architecture of our neural network was chosen for two primary reasons:

- The first reason was that using a compact architecture reduced the risk of overfitting. The OpenML Diabetes dataset contained only 768 samples, which is relatively small for training deep neural networks. During experimentation, we did achieve higher training accuracy with larger networks; however, their performance on the validation and test sets did not improve accordingly, a behaviour indicating that the models were memorizing the training data rather than learning the patterns, resulting in poor accuracy to unseen samples.
- The second reason was computational efficiency. Although larger networks have a greater capacity to model complex relationships, they require significantly more processing power and training time. Had we had a bigger

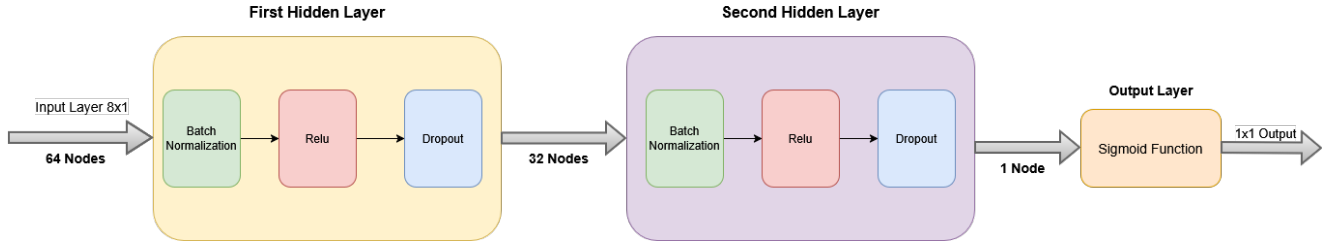


Fig. 1. Diagram of the neural network

dataset, increasing the network complexity would also increase the resources we needed for the FPGA hardware and for the training process.

C. Training

For the training process we used the following parameters and optimization strategies:

- **Loss Function:** Since diabetes detection is a binary classification problem, the network was optimized using **Binary Cross-Entropy** loss, defined as:

$$L = -\frac{1}{N} \sum_{i=1}^1 [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where y_i represents the true human-given label and p_i represents the predicted probability from the final sigmoid layer.

- **Optimizer:** The **Adam** (Adaptive Moment Estimation) optimizer adjusts the learning rates during training for each parameter automatically. An initial learning rate of $\eta = 0.001$ was selected.
- **Learning Rate Scheduling:** a **ReduceLROnPlateau** callback was implemented, because if the validation loss failed to improve for 10 consecutive epochs, the learning rate was dynamically scaled down by a factor of 0.5 down to a minimum floor of 10^{-5} .
- **Batch Size and Epochs:** The model was trained with a batch size of 32 over a maximum of 150 epochs.
- **Early Stopping:** To safeguard against overfitting and to reduce the training time an **EarlyStopping** callback monitored the validation loss. Training was terminated early if no improvement was observed for 20 consecutive epochs, at which point the best performing weights were restored.
- **Class Weighting Application:** As mentioned earlier, the computed class weights (1.37 for the positive class and 0.78 for the negative class) were fed directly into the `fit()` method to scale the loss updates and counter the 2 : 1 data imbalance.

D. Optimizations

1) **Reuse Factor:** The defining advantage of FPGAs over sequential processors is the ability to physically lay out hundreds of parallel operations in silicon. However, an FPGA

has a finite number of physical resources.

The Resource-Latency Tradeoff

When computing the output of a neural network layer, the mathematical operation is Matrix Multiplication, essentially a series of nested loops.

- **Fully Unrolled:** If the loop is completely unrolled, every single multiplication happens simultaneously in one clock cycle. This provides the absolute lowest latency but requires a massive amount of physical multipliers, which easily exceeds the capacity of edge FPGAs.
- **Fully Rolled:** If the loop is executed sequentially, only one multiplier is instantiated in hardware, and it is reused for every calculation. This consumes minimal resources but causes latency to skyrocket and for our case wouldn't be practical.

The Reuse Factor

In hls4ml, this architectural tradeoff is explicitly controlled via the **ReuseFactor** parameter. The **ReuseFactor** defines exactly how many times a single physical multiplier is reused per inference.

Setting a **ReuseFactor** of 16 dictates that the HLS compiler will instantiate enough parallel logic to compute the layer's math, while time-multiplexing each physical DSP block 16 times.

Hardware Impact

By tuning this parameter, we can balance the architecture to ensure our model fits the target silicon without over-allocating DSPs, all while still achieving low latency.

2) **Fixed-Point Representation:** Standard neural networks are trained using 32-bit floating-point arithmetic. While FP32 provides exceptional dynamic range and precision, floating-point math units are notoriously expensive to implement on FPGAs in terms of area, power consumption, and clock routing.

To bypass this, hardware-accelerated models utilize fixed-point arithmetic, denoted in HLS as `ap_fixed<W, I>`.

- **Word Length (W):** The total number of bits allocated to represent the number.

- **Integer Length (I):** The number of bits allocated to the integer portion, including the sign bit. The remaining bits ($W - I$) are dedicated to the fractional part.

By restricting the model to highly quantized states, the memory footprint of the model is drastically reduced.

3) *Pruning sparse layers:* In standard deep learning frameworks, dense layers consist of fully connected neurons where every input is multiplied by a corresponding weight. While this helps the model converge during training, it creates a massive computational bottleneck for hardware deployment.

Magnitude pruning

Magnitude pruning is a technique which systematically removes connections that contribute the least to the network’s final predictions. The core assumption is that weights with values close to zero have a negligible impact on the activation of subsequent neurons.

The Pruning Schedule

Instead of removing all targeted weights at once, a gradual approach like PolynomialDecay is often employed. This allows the network to adapt to the sudden loss of parameters by adjusting the remaining weights over several epochs, minimizing the degradation of the model’s accuracy.

Hardware Impact

In traditional CPU/GPU architectures, zero-valued weights still consume memory bandwidth and compute cycles unless specialized sparse-matrix libraries are used. However, using High-Level Synthesis tools like hls4ml, these zeros are treated as constants and as a result the C++ compiler optimizes out the multiplication operations associated with zero-weights. This physically eliminates the routing, LUTs and DSP blocks that would have been instantiated for those operations, freeing up resources on the FPGA.

4) *Quantization-Aware Training via QKeras:* Simply truncating a model from 32-bit to 4-bit precision post-training usually results in catastrophic accuracy loss. To combat this, Quantization-Aware Training is employed. Using the QKeras library, the model simulates the restricted precision during the forward pass of the training phase. The optimizer calculates the gradients using standard precision but updates the “fake-quantized” weights. This allows the neural network to learn a parameter distribution that inherently respects the strict hardware boundaries, bridging the gap between software simulation accuracy and on-chip hardware accuracy.

Hardware impact

Reducing the precision yields cascading benefits for FPGA deployment:

- **BRAM Reduction:** The storage requirements for weight matrices plummet, allowing larger models to fit entirely within the FPGA’s ultra-fast internal Block RAM rather than relying on slow, external DDR memory.

- **Logic Optimization:** Multipliers for 4-bit and 6-bit numbers are vastly smaller than 32-bit multipliers. Furthermore, synthesis tools can often map these ultra-low precision multiplications entirely into standard LUTs, preserving the scarce, dedicated DSP slices for more complex operations.

V. HARDWARE IMPLEMENTATION WITH HLS4ML

After we had successfully trained and optimized our neural network in software using Keras and QKeras, the next step was deploying our model onto hardware. So for this task, we utilized hls4ml, an open-source framework designed to translate trained machine learning models into High-Level Synthesis C++ code, which can then be synthesized for FPGAs.

A. Model Conversion and Configuration

The transition from the software model to a hardware description required a specific configuration to balance our latency, throughput, and hardware resource utilization. The hls4ml tool allows us to extract the weights, biases, and architecture from the files generated by Keras and map them directly to hardware structures.

We generated a custom configuration dictionary using `hls4ml.utils.config_from_keras_model`. To align with our target hardware (the Xilinx Alveo U250 board) and the following settings were applied:

- **Backend:** Vitis HLS was selected as the synthesis back-end.
- **Clock Period:** We specified a target clock period of 5.0 ns.
- **IO Type:** The `io_parallel` interface was utilized to maximize throughput by computing the entire inference in a single, unrolled pipeline where all inputs are processed concurrently.
- **Strategy:** The `Latency` optimization strategy was chosen. Because our model is relatively small, we prioritized execution speed over resource sharing, allowing Vitis HLS to fully unroll the loops.

Hls4ml natively supports the quantized QKeras datatypes, which we defined and explained earlier. It maps our fractional 4-bit weights and activations (`quantized_bits(4, 1)`) directly into `ap_fixed<4,1>` arbitrary precision C++ types. This prevented the compiler from defaulting to the heavy 32-bit floating-point DSP operations thus improving resource utilization and latency.

B. C-Simulation and Logic Synthesis

Once the HLS code was generated, the workflow proceeds through two main phases: C-Simulation and C-Synthesis.

- 1) **C-Simulation:** Before committing to logic synthesis, and to save time in case something fails, we first run a C-Simulation using a test bench generated by hls4ml. During this process we feed the original testing dataset through the generated C++ model to verify that the mathematical outputs of the HLS design

exactly match the predictions of the QKeras software model. As expected, our hardware model reproduced the same accuracy, validating the bit-accurate behavior of the fixed-point arithmetic.

- 2) **C-Synthesis:** Finally, the C++ code was synthesized into Register Transfer Level (RTL) code using Vitis HLS. During this stage, the compiler maps the mathematical operations to actual FPGA primitives (Look-Up Tables, Flip-Flops, DSP slices, and Block RAMs).

VI. RESULTS AND EVALUATION

To evaluate the impact of quantization and pruning on both model performance and hardware resource utilization, we implemented and compared four different neural network configurations.

- **Model 1 (Baseline):** Unquantized and unpruned.
- **Model 2:** Quantized and unpruned.
- **Model 3:** Unquantized and pruned.
- **Model 4:** Quantized and pruned.

The performance, latency, and hardware efficiency of our models were evaluated by comparing them to the standard baseline configuration against the other pruned and quantized models.

1) Model 1 (Baseline) Hardware Characteristics:

- **Configuration:** Standard precision parameters, `ReuseFactor` of 32.
- **Resource Utilization:** For the baseline, although the resources we required did not exceed the ones we had, the unoptimized version had a slack of almost 1 ns since the estimated clock period was not enough for the critical path to execute. The baseline usage was as follows:

Metric	Value
Target Clock Period	5.00 ns
Estimated Clock Period	3.647 ns
Best Latency	91 cycles
Worst Latency	91 cycles
Initiation Interval (II)	31 cycles
BRAM	1
DSP	83
Flip-Flops (FF)	24115
Look-Up Tables (LUT)	73157
URAM	0

TABLE II

HLS SYNTHESIS RESULTS FOR MODEL 1 (UNQUANTIZED, UNPRUNED).

2) Model 2 Hardware Characteristics:

- **Configuration:** For the precision parameters we maintained an almost standard precision where we tested 4 bits for integer and 16 for the floating point to test whether our accuracy would stay the same or plummet with the removal of 12 bits. The `hls4ml` converter was also configured with a `ReuseFactor` of 32.
- **Resource Utilization:** The synthesis results show that the generated hardware design satisfied the required timing constraints and we did not have

any slack like the baseline model. Although the target clock period was set to 5.00 ns, the estimated clock period after synthesis is 3.65 ns, indicating that we could operate the design at a higher clock frequency than required. Furthermore, the implementation achieved a latency of 29 clock cycles with an initiation interval (II) of 1, meaning that a new input can be accepted every clock cycle while previous inputs were still being processed. As shown in the synthesis results, the utilization of all hardware resources was within the available limits of the target, since our selected FPGA provided sufficient resources to implement our neural network without exceeding its hardware constraints.

Metric	Value
Target Clock Period	5.00 ns
Estimated Clock Period	3.65 ns
Best Latency	29 cycles
Worst Latency	29 cycles
Initiation Interval (II)	1 cycle
BRAM	1
DSP	1487
Flip-Flops (FF)	110390
Look-Up Tables (LUT)	48818
URAM	0

TABLE III

HLS SYNTHESIS RESULTS FOR MODEL 2 (QUANTIZED, UNPRUNED).

3) Model 3 Hardware Characteristics:

- **Configuration:** Standard precision parameters, `ReuseFactor` of 16 and a pruning with final sparsity 50%
- **Resource Utilization:** Although the estimated clock period rised,as well as the latency in cycles it was still within operational limits. Apart from that, the usage fell significantly.

Metric	Value
Target Clock Period	5.00 ns
Estimated Clock Period	4.660 ns
Best Latency	71 cycles
Worst Latency	71 cycles
Initiation Interval (II)	16 cycles
BRAM	1
DSP	85
Flip-Flops (FF)	35,836
Look-Up Tables (LUT)	39,201
URAM	0

TABLE IV

HLS SYNTHESIS RESULTS FOR MODEL 3(UNQUANTIZED, PRUNED) .

4) Model 4 Hardware Characteristics:

- **Configuration:** Synthesized from the 6-bit QKeras architecture with the `hls4ml` converter set to a `ReuseFactor` of 16, a pruning of 0.5 and a global precision of `ap_fixed<6,3>`.
- **Resource Utilization:** The reduced bit-width allowed our model to completely ignore DSP usage, registering 0 DSP units. It also required far fewer logic resources, utilizing only 1 BRAM_18K block,

1,275 FFs, and 12,306 LUTs. Also the estimated clock period dropped again to 3ns as well as the best latency and II which could allow us to up the frequency even more if it was needed.

Metric	Value
Target Clock Period	5.00 ns
Estimated Clock Period	3.180 ns
Best Latency	6 cycles
Worst Latency	6 cycles
Initiation Interval (II)	1 cycle
BRAM	1
DSP	0
Flip-Flops (FF)	1,275
Look-Up Tables (LUT)	12,306
URAM	0

TABLE V
HLS SYNTHESIS RESULTS FOR MODEL 4(QUANTIZED, PRUNED).

A. ROC Curve Evaluation Across Deployment Stages

To evaluate the impact of compression, quantization, and hardware synthesis on performance, we used Receiver Operating Characteristic (ROC) curves across four distinct stages of the optimization pipeline. The Area Under the Curve (AUC) served as the metric to evaluate performance as the network was step by step transitioning from standard floating-point neural network to a FPGA-deployable hardware architecture.

1) *Model 1 (Unoptimized Baseline Model)*: The ROC curve for the baseline model represents the performance upper bound of the architecture, operating with full floating-point precision and unpruned connections. This configuration achieves the highest AUC, as the network retains maximal capacity to distinguish between the diabetic and non diabetic patients.

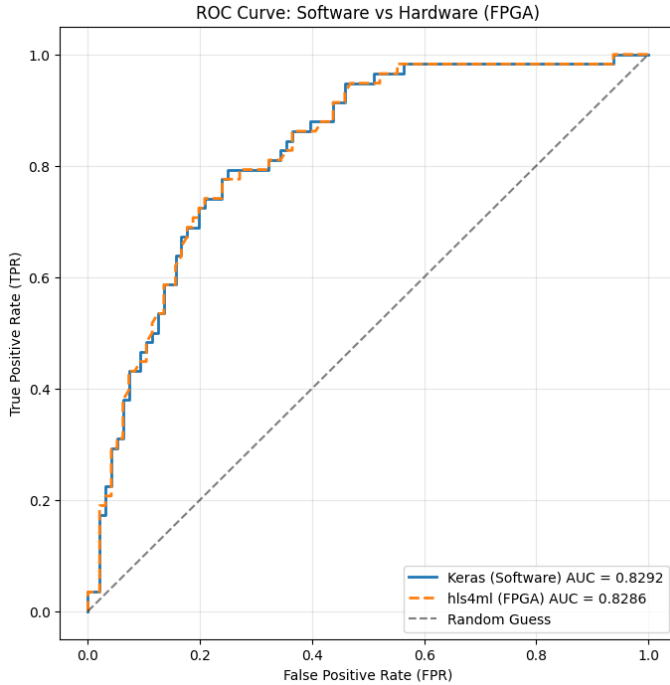


Fig. 2. Roc curve for model 1

2) *Model 2 (Quantized Keras Model)*: The Area Under the Curve for the synthesized hls4ml hardware implementation has a negligible degradation of less than 1% compared to the unquantized counterpart. This minimal variance helped us confirm that the transition to an FPGA-deployable fixed-point architecture preserves the network's performance and the hardware model maintains its sensitivity without compromising its classification reliability.

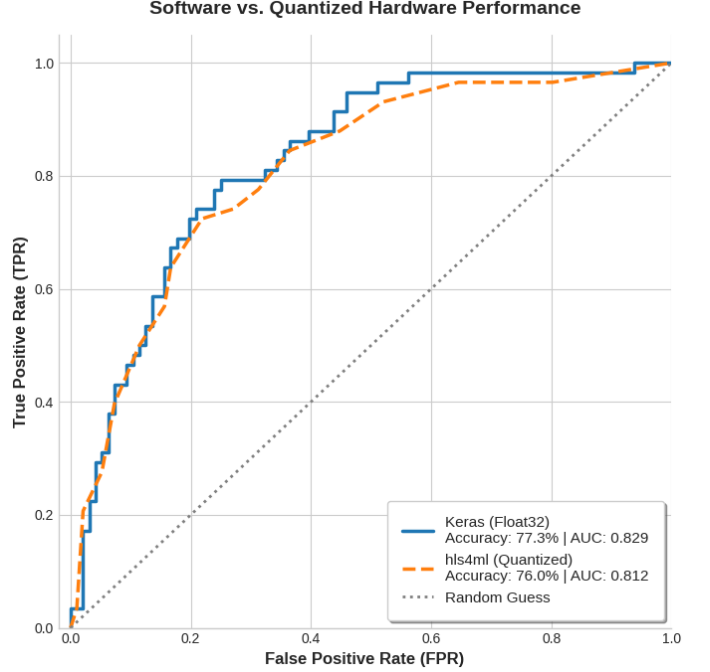


Fig. 3. Roc curve for model 2

3) *Model 3 (Pruned Keras Model)*: Following the introduction of the polynomial decay pruning schedule (`final_sparsity=0.5`), the near-identical AUC confirms that eliminating 50% of the network's weights removes redundant parameters without degrading our model's underlying sensitivity and accuracy.

4) *Model 4 (Quantization-Aware Pruned Keras Model)*:: The ROC curve for Model 4 reflects the combined results of 50% sparsity and 6-bit fixed-point quantization implemented via QKeras (`quantized_bits(6, 2)`). A minor divergence from the Model 1 and Model 2 ROC curves is observed, however, because the network is trained with Quantization-Aware Training (QAT), the weights adapted, resulting in a decent AUC and preserving its overall diagnostic power.

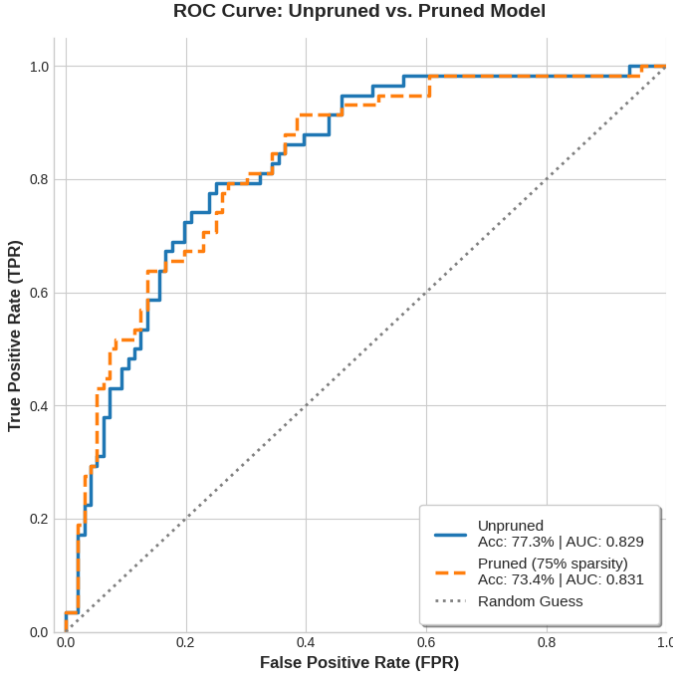


Fig. 4. Roc curve for model 3

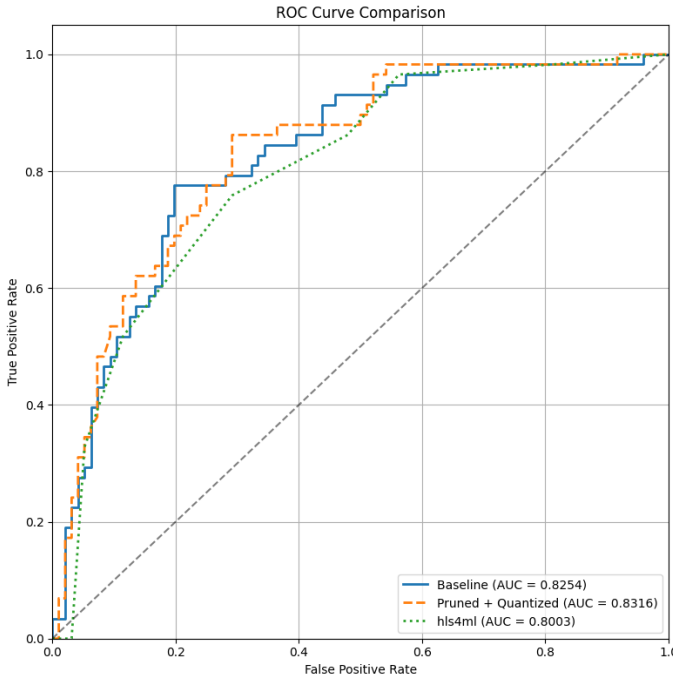


Fig. 5. Roc curve for model 4

Summary: Evaluating the four ROC curves illustrates a highly favorable trade-off. While shifting from Model 1 to Model 4 results in a minor degradation in AUC, this fractional loss in classification certainty is heavily outweighed by the hardware efficiency gains achieved: a 50% reduction in memory footprint, the total elimination of DSP multiplier

usage (0 DSPs), and an inference latency reduction down to 6 clock cycles (30.00 ns).

VII. VIVADO – PPA ANALYSIS

A. Overview

This report details the power consumption and resource utilization of a hardware-accelerated neural network designed for diabetes detection. The model was implemented on an Alveo U250 FPGA using a quantized precision of 6-bit fixed-point arithmetic and a 50% magnitude pruning schedule. The timing constraints for this design were fully met, achieving a positive Worst Negative Slack of 4.278 ns.

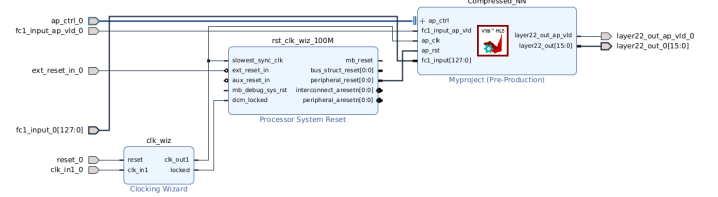


Fig. 6. Block diagram for vivado synthesis and implementation

B. On-Chip Power Summary

The total on-chip power consumption is estimated at 4.157 W. The majority of this power footprint is drawn by the physical device's static operations.

Power Category	Power (W)	Percentage
Total On-Chip Power	4.157 W	100%
Device Static Power	2.966 W	71%
Dynamic Power	1.191 W	29%

TABLE VI
POWER ANALYSIS OF COMPRESSED NN

Thermal Metrics: The junction temperature is estimated at 27.2°C, providing a wide thermal margin of 72.8°C at an ambient temperature of 25.0°C.

C. Dynamic Power Breakdown

The active processing of the neural network draws 1.191 W of dynamic power, which is distributed across the FPGA's physical resources as follows:

- Logic: 0.586 W (49%)
- Signals: 0.432 W (36%)
- MMCM (Mixed-Mode Clock Manager): 0.097 W (8%)
- Clocks: 0.062 W (5%)
- I/O: 0.014 W (1%)
- BRAM: < 0.001 W (< 1%)

D. Resource Utilization and Power Correlation

The power analysis aligns closely with the architectural optimizations applied during the model's conversion to hardware:

- **DSP-Free Architecture:** The design achieves its processing targets utilizing exactly 0 DSP slices. Because 50% of the weights were pruned and the remaining data was quantized to an ultra-low precision, the synthesis tool successfully mapped all multiplication operations entirely to standard Look-Up Tables.
- **Logic Power Dominance:** Because the math operations were shifted away from DSPs, standard logic blocks account for the bulk of the resource utilization, specifically consuming 38,073 CLB LUTs and 10,710 CLB Registers. Consequently, the "Logic" and "Signals" categories combined account for 85% of the total dynamic power budget.
- **Low Memory Power:** The highly optimized memory footprint of the compressed model utilizes only a single Block RAM tile. This directly correlates with the minimal dynamic power (<0.001 W) consumed by BRAM operations.

VIII. CONCLUSION

Latency and Speed

- The baseline model requires a best and worst latency of 91 cycles.
- The optimized model reduces this latency drastically down to just 6 cycles.
- The baseline model has an Initiation Interval (II) of 31 cycles.
- The optimized model achieves an II of 1 cycle.
- An II of 1 cycle indicates the optimized architecture can accept a new input every single clock cycle.

While both architectures target a 5.00 ns clock period, the estimated clock period improves from 3.647 ns in the baseline to 3.180 ns in the optimized version.

Resource Utilization

- The unoptimized baseline utilizes a significant amount of FPGA resources, requiring 83 DSPs, 24,115 Flip-Flops (FF), and 73,157 Look-Up Tables (LUT).
- The optimized model completely eliminates the need for dedicated math blocks, utilizing 0 DSP units.
- The optimized model severely cuts logic resource requirements, using only 1,275 FFs and 12,306 LUTs.
- Both models maintain a highly efficient memory footprint by utilizing exactly 1 BRAM and 0 URAM.

As summarized in our report evaluation, shifting to the optimized model yields a highly favorable trade-off: the total elimination of DSP usage and a massive drop in inference latency heavily outweigh the fractional degradation in the model's classification certainty.

METHODOLOGY AND TOOLCHAIN REFERENCES

The implementation, optimization, and synthesis software used and other several foundational frameworks and open-source toolchains:

- **Hardware Translation (hls4ml):** The conversion of pre-trained Keras models into synthesizable C++ and High-Level Synthesis (HLS) code was executed using the hls4ml toolchain. [?].
- **Quantization-Aware Training (QKeras):** Low-precision fixed-point quantization was implemented via QKeras, an extension of the Keras ecosystem that natively supports Quantization-Aware Training (QAT).
- **Magnitude-Based Pruning (TF-MOT):** This offered the methodologies we used for eliminating redundant neural connections to reduce hardware memory footprints.
- **Synthesis Backend (Vitis HLS):** Hardware resource estimation, latency profiling, and C-simulation (csim) were compiled using the AMD/Xilinx Vitis HLS backend, which synthesizes the generated C++ designs into RTL instructions targeted for FPGA architectures.

REFERENCES

- [1] hls4ml tutorial. Available: <https://github.com/fastmachinelearning/hls4ml-tutorial>.
- [2] OpenML diabetes dataset. Available: https://www.openml.org/search?type=data&sort=nr_of_likes&status=active&id=37